B

Cost & safety

> **Note**
>
> *Three things you can't learn from the chapters: a rough sense of what your staff should cost per month, the symptoms of every rate-limit you're likely to hit, and a one-page map of where your logs and traces actually live.*

This appendix gives you a working operator's view of cost, throttling, and observability: enough to recognize a problem when you see it, and enough to ask Claude the right diagnostic question. The numbers are ranges, never singletons. Anthropic's pricing changes; vendor rate limits move; log paths get tweaked between releases. The pricing page Anthropic publishes is canonical; this appendix points there for current rates and stays focused on shape.

Section 1: Token budgets by agent type

Every agent in the book has a different cost shape. A triage agent that reads a subject line and a one-line preview costs almost nothing per invocation. A newsroom that ingests a hundred-line research dump and produces a thousand-word brief costs roughly two orders of magnitude more. The table below is for budgeting intuition: *order of magnitude*, not invoice prediction.

Agent archetype (chapter)	Input tokens / call	Output tokens / call	Calls / day (typical)	Monthly cost band*
Inbox triage: Iris (*Chapter 11*)	small	small	many (per email)	low
Morning brief reader (*Chapter 9*)	medium	small	once	low
Reuben reconciliation (*Chapter 13*)	medium	medium	weekly	low–medium
Drafter (Iris draft mode, *Chapter 11*)	medium	medium	per draft	medium

Agent archetype (chapter)	Input tokens / call	Output tokens / call	Calls / day (typical)	Monthly cost band*
Research analyst / code-researcher (*Chapter 7*)	large	medium	on demand	medium–high
Newsroom editor (*Chapter 18*)	large	large	per piece	high
Fact-checker (*Chapter 18*)	large	small	per piece	medium
Watchdog (*Chapter 20*)	small	small	scheduled, frequent	low
Custom MCP host (*Chapter 16*)	varies	small	on demand	low–medium

* Bands assume a single operator using each agent at the cadence the chapters establish. Multiply by your actual invocation rate. The Anthropic pricing page has current per-token rates; this table is shapes, not dollars.

Three sample monthly bills

The author's own bills, anonymized, dated to two billing cycles before press.

Light Maya: Iris and a morning brief. A solo operator running Iris on a personal inbox plus the *Chapter 9* morning brief. Maybe 200 Iris invocations a day, one brief, no scheduled research. Bill lands in the *I-pay-more-for-coffee* range. The dominant cost is Iris's per-email inputs, not the brief.

Medium Devon: Iris + Reuben + newsroom 2x/week. A small business owner with Iris running on customer mail, Reuben running every Friday, newsroom firing twice a week to draft client updates. Bill lands in the *cheaper-than-a-mid-tier-SaaS* range. The dominant cost is the newsroom; Reuben is rounding error; Iris is steady.

Heavy author: full mesh, watchdog firing nightly, scheduled research. All seventeen artifact chapters' worth of agents, plus three custom MCPs, plus a research subagent that runs nightly on five topics. Bill lands in the *real line item* range: comparable to a developer-tier SaaS subscription. Watchdog cost is invisible; research cost is half the bill; newsroom is the other half.

In every case, *the agents I worried about most cost least, and the ones I forgot about cost most.* The lesson is in *Section 2*.

What blows up your bill

I once left a `SessionStart` hook that re-ran a 200-line `CLAUDE.md` plus a Gmail MCP fetch every time I opened a new tab. That month's bill was real. The four ways an operator-scale staff suddenly costs more than expected:

- **Long CLAUDE.md re-loaded every session.** Every new `claude` invocation reads the file. If you've stuffed `CLAUDE.md` with three years of history, every session pays for it.

- **MCPs that return enormous payloads** (the chunk of data a tool hands back). Gmail's full-thread mode, Slack's full-channel digest, Notion's full-database export. Configure for the smallest payload that answers the question.
- **Subagents that spawn subagents.** A drafter that calls a research agent that calls a fact-checker, without a depth cap, one input can fan out into a dozen invocations. Limit recursion in agent prompts; the watchdog from *Chapter 20* catches the symptom (sudden cost spike), not the cause.
- **Hooks that fire on every prompt instead of on session boundaries.** A `PreToolUse` hook on `Bash` is supposed to be cheap; if you accidentally wrote one that calls Claude itself for a permission check, every Bash call pays the round-trip.

Run job 25 from *Appendix A* (the cost-trend check) every Sunday during the governance hour. Bills bend before they break.

Section 2: Rate limits: what you see, what it means, what to do

Rate limits arrive as five distinct symptoms. Sorting which one you've hit is the first step.

What you see	What it means	What to do
`429 Too Many Requests` from Anthropic	Per-minute throttle on the API. Burst exceeded the cap.	Wait. The error usually includes a retry-after header. Claude Code already backs off on this; if you're scripting around it, mirror that.
`You've reached your daily limit` (in the CLI)	Per-day quota wall on your Claude plan.	Either wait for the daily reset (the message shows when), upgrade the plan, or batch the remaining work into the next window. The "use the pause to think about what's actually worth asking" rule from v1 still applies.
Gmail / Slack / Notion MCP returns "rate limit exceeded"	The vendor (not Anthropic) throttled the MCP's call.	Each vendor publishes its own limits: check theirs. Gmail has per-user-per-second caps on `messages.send`; Slack rate-limits posting per channel; Notion rate-limits per integration. Stagger calls; don't queue 50 sends at 9 AM.

What you see	What it means	What to do
Silent slowness: no error, just delay	An MCP server is backing off internally without surfacing the error.	Tail the MCP server log (Section 3). The backoff usually shows up as repeated retries. If you can't find the log, smoke-test the MCP from a scratch session: the failure mode is more visible there.
A scheduled job ran but produced empty output	Often a transient throttle that the agent caught and swallowed.	Add an exit-non-zero condition to the agent's verify script (Appendix A job 14). The watchdog (Chapter 20) will catch the next occurrence.

Backoff: who handles it, and where you have to

Claude Code retries 429s and recoverable network errors transparently. Anthropic's API client honors the standard headers. *You* have to handle:

- **Vendor MCPs that don't.** Some community MCPs treat a 429 from Gmail or Slack as a hard failure. The fix is one prompt to Claude: *"the [MCP] is failing on rate limits, wrap the failing call in a retry with exponential backoff capped at 30 seconds."* (Exponential backoff means waiting longer between each retry (one second, then two, then four) instead of hammering the service.)
- **Hook scripts that call vendor APIs directly.** Hook scripts run outside Claude's normal retry path. If your `SessionStart` hook curls a Notion endpoint, you write the backoff or you accept the brittleness.
- **Scheduled jobs stacked at the same minute.** Don't fire 10 things at 6:30 AM. Stagger by 5 minutes. The "rate-limit-aware schedule" pattern is one diagonal line in the cron table, not a horizontal one.

When in doubt, paste the rate-limit error into Claude with: *"this came from [SERVICE]. What's the actual cap and what's the right backoff?"* Half the time Claude knows the cap from the vendor's docs. The other half you check the docs together.

Section 3: Observability: where logs live, what to grep

(*grep* is the Unix tool that searches text for a pattern: you point it at a file and it prints the lines that match.)

Three paths cover roughly 90% of what you'll need to look at. The rest is documented in Anthropic's published observability reference; treat that as canonical for current paths.

What you want	Where it lives	How to look
The transcript of the most recent session	`~/.claude/projects/<project-hash>/` (one folder per project; file names are session IDs, stored as JSONL shards)	`claude --resume` opens the interactive picker: the on-disk format is designed for resume, not casual `cat`-and-read; use `claude rm <id>` to remove a specific session
Hook stdout/ stderr (the normal output and error streams a program prints)	Wherever the hook redirected it: best practice is `~/.claude/logs/hooks/[hookname].log`	`tail -F ~/.claude/logs/hooks/[hookname].log` while the hook fires
Connector / MCP status	`/mcp` shows each connector's status, the account it's using, and its tools: the place to look first. A server you built yourself logs wherever you make it log.	Open `/mcp` in a session; for a self-built server, `tail -F` the log path you set (e.g. `~/.claude/logs/mcp/[name].log`)
Scheduled-job runs	The job's own log path (you set it when authoring the job; if you didn't, it went to the system mail)	`tail -F ~/work/logs/[job-name].log` if you set the convention
The `.claude/` change history	Git, if you committed the folder; otherwise filesystem mtime	`git log --oneline -- .claude/` or `ls -lt .claude/`

The convention I run, and recommend: every hook and every scheduled job writes to a file under `~/.claude/logs/` with a name that matches the hook or job. The *Chapter 9* SessionStart hook logs to `~/.claude/logs/hooks/sessionstart.log`. Reuben logs to `~/.claude/logs/jobs/reuben.log`. If you let Claude author the hook (*Appendix A* job 12), tell it to set the log path explicitly. Default behavior of a freshly authored hook is to drop output into the void.

Three grep recipes I use most

"Did my hook fire this morning?"

```
tail -100 ~/.claude/logs/hooks/sessionstart.log | grep "$(date +%Y-%m-%d)"
```

"Why did Reuben exit with code 1 last Friday?"

```
grep -B 5 "exit 1" ~/.claude/logs/jobs/reuben.log | tail -50
```

"Why did my custom MCP return an empty payload at 8:42?" (a server you built logs where you tell it to: see *Chapter 16*; for a connector, check `/mcp` instead)

```
grep -A 10 "08:42" ~/.claude/logs/mcp/personal-data.log
```

The "tail-while-it-happens" pattern catches things grep can't. Open a second terminal pane, run `tail -F` on the suspect log, then trigger the agent in the first pane. The lines that appear when you run the agent are the ones that matter; everything before is noise.

When the log isn't enough, paste the last 50 lines to Claude with: *"this log is from [AGENT], diagnose why it [SYMPTOM]."* You'll get a reading and usually a fix. If the diagnosis points at a specific line, follow up with job 19 from *Appendix A*.

A closing note on numbers in this appendix

Everything here is shape, not number. The Anthropic pricing page is authoritative for current per-token rates. Vendor rate-limit pages are authoritative for vendor caps. The published observability reference is authoritative for the current log layout. Bookmark those three pages; the errata for this book carries the major deltas between print runs but does not try to replace the live sources.

The cost-trend job (*Appendix A*, job 25) is the one I run most. Once a week, on a quiet Sunday, against the previous 30 days. Cheap insurance against the bill that surprises.